Logging, Auditoria e Observabilidade Básica

Programação para a Web 2

Aula 15

Evandro J.R. Silva

Introdução

- Em aplicações *backend*, especialmente em produção, o sistema precisa "contar a própria história". Sem registros adequados, é praticamente impossível:
  - Descobrir por que um erro ocorreu;
  - Reconstruir o que um usuário fez;
  - Atender requisitos de auditoria e conformidade (LGPD, PCI DSS – *Payment Card Industry Data Security Standard*, etc.);
  - Entender o comportamento do sistema sob carga.
- Para isso precisamos de Logging, Auditoria e Observabilidade.

Introdução

- ***Logging***
  - É o registro cronológico de eventos que ocorreram na aplicação.
- **Auditoria**
  - É o uso desses registros (ou de registros específicos) para responder “quem fez o quê, quando e com qual resultado”, normalmente com foco em segurança, conformidade e responsabilidade.
- **Observabilidade básica**
  - Começa quando conseguimos correlacionar eventos de uma mesma requisição e extrair informações úteis sem precisar reproduzir o problema.

O que registrar e por quê

Principais categorias de eventos

Categoria	Exemplos	Por quê registrar
Erros e exceções	Falha de conexão com banco, *timeout* de API externa, erro inesperado.	Debugging e diagnóstico
Eventos de negócios	Pedido criado, pagamento confirmado, usuário cadastrado.	Monitoramento de negócio e auditoria
Ações de segurança	Tentativa de login (sucesso/falha), mudança de permissão, acesso a recurso sensível.	Segurança e auditoria
Operações crísticas	Alteração de dados importantes, exclusão, exportação.	Trilha de auditoria
Performance	Duração de requisições, tempo de consultas lentas	Identificação de gargalos
Ciclo de vida	Inicialização do serviço, *shutdown*, deploy	Contexto operacional

O que registrar e por quê

- ***Logging* operacional × Auditoria**
  - **Log operacional:** focado em diagnóstico técnico. Pode ser mais verboso, ter retenção menor e ser amostrado em alguns casos. Serve principalmente para desenvolvedores e operações.
  - **Log de auditoria:** focado em "quem fez o quê". Deve ser mais estável, ter retenção definida (dias, meses ou anos conforme o risco/legal), não deve ser amostrado e idealmente ter controle de integridade e acesso restrito.
- Exemplo de pergunta respondida por cada tipo:
  - Operacional: "Por que a API de pagamento retornou `500` às 14:32?"
  - Auditoria: "Quem alterou o perfil do usuário 48291 às 14:32 e de onde veio a requisição?

O que registrar e por quê

- **O que nunca registrar**
  - Senhas, tokens, chaves de API, cookies de sessão;
  - Números de cartão de crédito ou dados bancários completos;
  - Dados pessoais sensíveis desnecessários (CPF completo, dados de saúde, etc.);
  - Corpo completo de requisições/respostas em produção (a menos que seja sanitizado).
- Sempre sanitizar ou mascarar o que for necessário (ex.: últimos 4 dígitos do cartão, e-mail parcialmente mascarado).

Níveis de log

- Os níveis permitem controlar o volume de informação e filtrar mensagens de acordo com a gravidade ou relevância.
  - **Regra prática em produção:** nível mínimo geralmente tem `INFO`. `DEBUG` deve ser desligado ou ativado temporariamente para investigação específica.

Nível	Uso recomendado	Evitar usar para	Ambiente típico
`DEBUG`	Detalhes internos úteis para investigação (estado de variáveis, fluxo)	Produção contínua	Desenvolvimento/ Investigação pontual
`INFO`	Eventos normais e relevantes (pedido criado, requisição finalizada, serviço iniciado)	Cada passo interno ou log excessivamente verboso	Produção (padrão)
`WARN`	Situações anormais mas recuperáveis (*retry*, *fallback*, validação suspeita).	Condições esperadas	Produção
`ERROR`	Falhas que impactam uma operação específica e precisam de atenção	Validação de entrada do cliente (geralmente `WARN` ou `INFO`)	Produção
`FATAL/CRITICAL`	Falhas graves que podem derrubar o serviço ou corromper dados	Erros de requisição individual	Produção

Rastreabilidade de requisições

- Quando uma requisição atravessa várias camadas ou serviços, os logs ficam espalhados. Sem um identificador comum, é muito difícil reconstruir o que aconteceu.
- Os recursos e técnicas a seguir servem para esse propósito.

Rastreabilidade de requisições

- Request ID / Correlation ID
  - É um identificador único gerado no ponto de entrada da requisição (*gateweay*, *load balancer* ou primeiro serviço).
  - Deve ser incluído em todos os logs gerados durante o processamento daquela requisição.
  - É propagado para serviços *downstream* via *header* HTTP (normalmente `X-Request-ID` ou `X-Correlation-ID`).
  - Permite filtrar todos os logs de uma única requisição com uma única busca.

Rastreabilidade de requisições

- Logs estruturados (*structured logging*)
  - Em vez de mensagens de texto livro, usa-se logs em formato estruturado (JSON). Isso permite busca, filtragem e agregação eficientes. Exemplo:

```
{
  “timestamp”: “2026-10-13T18:35:12:123Z”,
  “level”: “info”,
  “message”: “Pedido criado com sucesso”,
  “service”: “order-service”,
  “environment”: “production”,
  “request_id”: “a3f8c2d1-4b21-7c22-9b6e-ee8e5b0b3f1a”,
  “user_id”: “usr_48921”,
  “order_id”: “ord_9182”,
  “duration_ms”: 47
}
```

Rastreabilidade de requisições

- Contexto de requisição
  - Em linguagens com modelo assíncrono (e.g., Node.js), o ideal é armazenar o `request_id` (e outros dados de contexto) em um armazenamento de contexto assíncrono, para que qualquer função chamada durante a requisição possa acessá-lo automaticamente, sem precisar passar o ID como parâmetro em toda a cadeia de chamadas.
- Trace ID
  - Em sistemas distribuídos mais avançados, usa-se o conceito de ***trace*** (rastreamento distribuído). Um `trace_id` identifica toda a jornada da requisição através de múltiplos serviços. Ferramentas como OpenTelemetry padronizam isso.

Exemplos práticos (Node.js + Express + Pino)

Logger básico estrututrado

```
// logger.js
const pino = require('pino');

const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  formatters: {
    level(label) {
      return { level: label };
    },
  },
  timestamp: pino.stdTimeFunctions.isoTime,
  base: {
    service: 'order-service',
    environment: process.env.NODE_ENV || 'development',
  },
});

module.exports = logger;
```

Exemplos práticos (Node.js + Express + Pino)

Middleware de Correlation ID

```js
// correlationMiddleware.js
const { randomUUID } = require('crypto');
const { AsyncLocalStorage } = require('async_hooks');

const storage = new AsyncLocalStorage();

function correlationMiddleware(req, res, next) {
  const correlationId =
    req.headers['x-request-id'] ||
    req.headers['x-correlation-id'] ||
    randomUUID();

  res.setHeader('x-request-id', correlationId);

  storage.run({ correlationId, method: req.method, path: req.path }, () => {
    next();
  });
}

function getCorrelationId() {
  return storage.getStore()?.correlationId;
}

module.exports = { correlationMiddleware, getCorrelationId, storage };
```

Exemplos práticos (Node.js + Express + Pino)

Uso no logger e nas rotas

```
// app.js (trecho)
const express = require('express');
const logger = require('./logger');
const { correlationMiddleware, getCorrelationId } = require('./correlationMiddleware');

const app = express();
app.use(correlationMiddleware);

app.post('/orders', async (req, res) => {
  const requestLogger = logger.child({
    request_id: getCorrelationId(),
    user_id: req.user?.id,
  });

  requestLogger.info({ body: { items: req.body.items?.length } }, 'Iniciando criação de pedido');

  try {
    // lógica de negócio...
    requestLogger.info({ order_id: 'ord_123' }, 'Pedido criado com sucesso');
    res.status(201).json({ orderId: 'ord_123' });
  } catch (err) {
    requestLogger.error({ err }, 'Falha ao criar pedido');
    res.status(500).json({ error: 'Erro interno' });
  }
});
```